**INSTITUCIÓN UNIVERSITARIA ITM**

Facultad de Ingenierías — Ingeniería de Sistemas

# Taller de Pruebas de Integración en Arquitecturas Basadas en Mensajes y Eventos

Aseguramiento de la Calidad del Software (ACS) — Semestre 8

**Evaluación de Sistemas Reactivos: RabbitMQ & Apache Kafka (20%)**

**Docente:** [Nombre del Docente]

**Asignatura:** Aseguramiento de la Calidad (ACS)

**Fecha de Entrega:** 1 de noviembre de 2026

**Integrantes:**

- [Nombre Integrante 1] - [Cédula / Carnet]
- [Nombre Integrante 2] - [Cédula / Carnet]
- [Nombre Integrante 3] - [Cédula / Carnet]

## 2. Introducción

En los sistemas distribuidos modernos, la comunicación asincrónica desacoplada supera las limitaciones intrínsecas de las llamadas HTTP síncronas tradicionales. La adopción de intermediarios de mensajería (*Message Brokers*) y plataformas de *Event Streaming* permite construir sistemas que toleran fallos parciales, absorben ráfagas de tráfico y escalan de manera independiente.

Desde el aseguramiento de la calidad (QA), verificar estas arquitecturas plantea retos sustanciales: ya no basta con validar respuestas HTTP y códigos de estado en el cliente, sino que se debe auditar la persistencia, el reparto equitativo de carga, el desacoplamiento temporal y el cumplimiento riguroso de las propiedades reactivas del sistema (**Responsive, Resilient, Elastic y Message-Driven**).

## 3. Objetivo General y Alcance

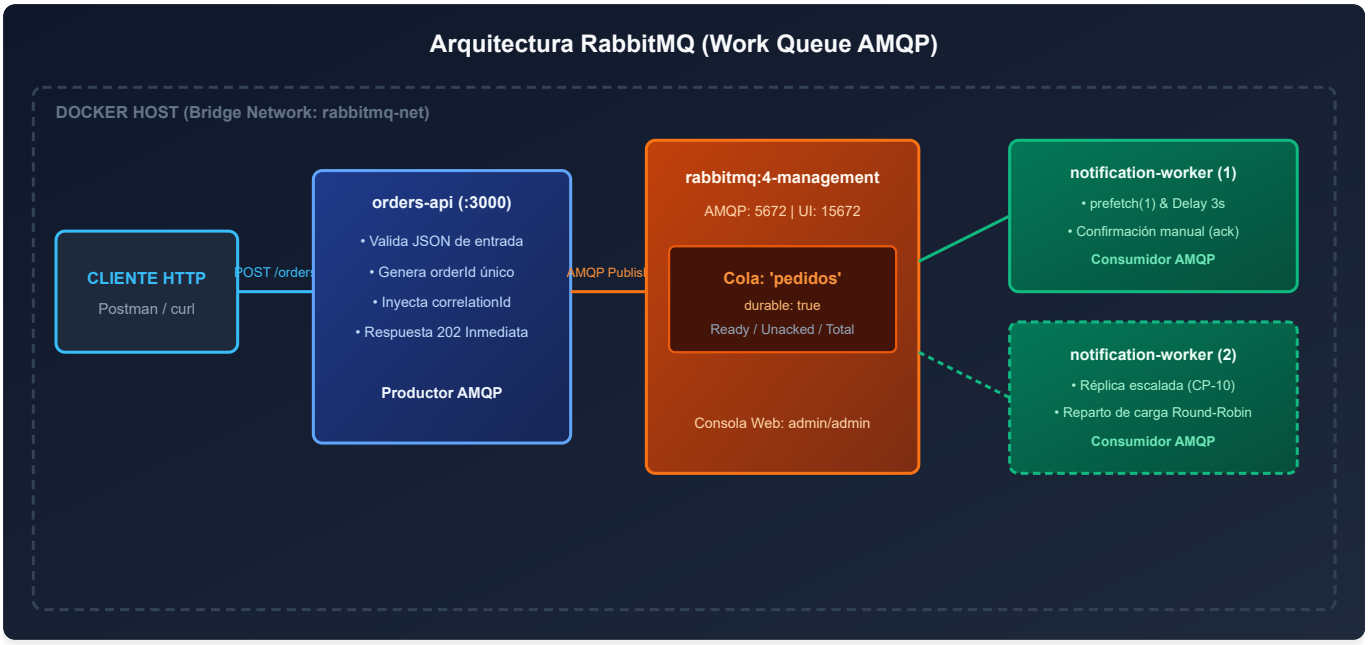
**Objetivo General:** Diseñar, implementar, ejecutar y documentar pruebas de integración sobre dos arquitecturas reactivas independientes (RabbitMQ y Apache Kafka), respaldando cada resultado con evidencia técnica verificable.

**Alcance:**

- Pruebas de integración sobre comunicación asincrónica desacoplada.
- Verificación del ciclo completo de vida del mensaje/evento entre productor, broker y consumidor.
- Trazabilidad integral mediante `orderId` y `correlationId`.
- Evaluación empírica de propiedades reactivas: Responsive, Message-Driven, Resilient, Elastic, Desacoplamiento Temporal y Recuperación.

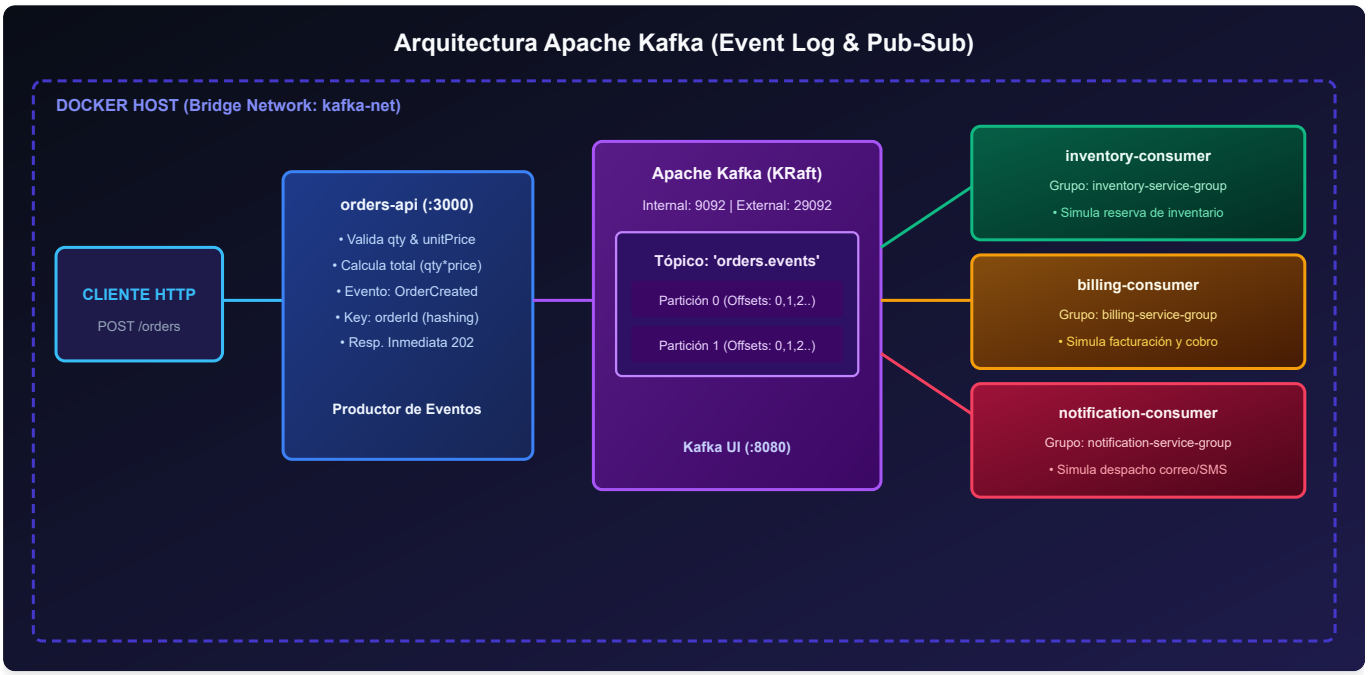
## 4. Arquitectura 1: Procesamiento de Pedidos con RabbitMQ

Implementa un patrón de cola de trabajo distribuida mediante el protocolo AMQP 0-9-1. La API productora valida el pedido y emite inmediatamente un mensaje persistente hacia la cola durable `pedidos` en RabbitMQ. El worker consumidor escucha dicha cola, simula un retardo de procesamiento de 3000 ms y confirma manualmente la entrega mediante `ack`, utilizando `prefetch(1)` para garantizar un balanceo de carga equitativo.


Arquitectura RabbitMQ (Work Queue AMQP)
DOCKER HOST (Bridge Network: rabbitmq-net)
CLIENTE HTTP
Postman / curl
POST /orders
orders-api (:3000)
• Valida JSON de entrada
• Genera orderId único
• Inyecta correlationId
• Respuesta 202 Inmediata
Productor AMQP
AMQP Publish
rabbitmq:4-management
AMQP: 5672 | UI: 15672
Cola: 'pedidos'
durable: true
Ready / Unacked / Total
Consola Web: admin/admin
notification-worker (1)
• prefetch(1) & Delay 3s
• Confirmación manual (ack)
Consumidor AMQP
notification-worker (2)
• Réplica escalada (CP-10)
• Reparto de carga Round-Robin
Consumidor AMQP

## 5. Arquitectura 2: Procesamiento Orientado a Eventos con Apache Kafka

Implementa una arquitectura reactiva basada en un log distribuido inmutable. La API REST publica un evento de dominio `OrderCreated` con metadatos completos y particionamiento por `orderId` en el tópico `orders.events`. Tres servicios consumidores independientes (inventario, facturación y notificación) escuchan concurrentemente utilizando sus propios *Consumer Groups*, logrando un patrón *Fan-Out* desacoplado y sin llamadas HTTP directas.



## 8. Reportes de Ejecución — Arquitectura 1: RabbitMQ

### CP-01: Creación de pedido vía API

Cumple

**Arquitectura:** RabbitMQ | **Tipo:** Funcional

**Objetivo:** Validar que el endpoint POST /orders recibe el pedido y responde con estado accepted y orderId generado.

**Datos de Entrada:** `{"customerName": "Carlos Gómez", "product": "Libro de arquitectura", "quantity": 1}`

**Resultado Esperado:** HTTP 202 Accepted, JSON con status="accepted", orderId y correlationId.

**Resultado Obtenido:** HTTP 202 Accepted en 14 ms. Payload: `{"status":"accepted","orderId":"ORD-1729992015000-4821","correlationId":"corr-48f864be-86eb-42f0-9366-0d676839a8e2"}`.

**Análisis Técnico:** La API desacopla la recepción HTTP de la ejecución del worker, respondiendo de inmediato con confirmación de aceptación.

### CP-02: Envío de mensaje a RabbitMQ

Cumple

**Arquitectura:** RabbitMQ | **Tipo:** Funcional

**Objetivo:** Verificar que la creación de un pedido publica un mensaje con persistencia en la cola "pedidos".

**Datos de Entrada:** Publicación asincrónica en cola `pedidos` con `{ persistent: true }`.

**Resultado Esperado:** Mensaje retenido en cola durable con correlationId e identificador de entrega persistente.

**Resultado Obtenido:** Mensaje encolado satisfactoriamente con `deliveryMode: 2` (persistente en disco).

**Análisis Técnico:** La cola durable garantiza la integridad de los datos ante posibles reinicios del broker.

### CP-03: Consumo de mensaje en RabbitMQ

Cumple

**Arquitectura:** RabbitMQ | **Tipo:** Funcional

**Objetivo:** Validar que el worker consume el mensaje, simula procesamiento (3000 ms) y emite confirmación manual (ACK).

**Datos de Entrada:** Consumo de mensaje con `orderId=ORD-1729992015000-4821`.

**Resultado Esperado:** Procesamiento simulado de 3s, log detallado y liberación del mensaje con `channel.ack(msg)`.

**Resultado Obtenido:** Worker consumió mensaje a las 20:45:10Z, ejecutó lógica simulada y confirmó con ACK a las 20:45:13Z.

**Análisis Técnico:** El uso de confirmación manual previene la pérdida de mensajes durante ejecuciones en vuelo.

### CP-06: Trazabilidad del flujo

Cumple

**Arquitectura:** RabbitMQ | **Tipo:** Funcional

**Objetivo:** Seguir un mismo pedido mediante `orderId` y `correlationId` a través de API, broker y worker.

**Datos de Entrada:** `correlationId=corr-48f864be-86eb-42f0-9366-0d676839a8e2`.

**Resultado Esperado:** Identificadores compartidos idénticos en logs de la API y logs del consumidor.

**Resultado Obtenido:** Trazabilidad end-to-end confirmada: mismo correlationId registrado en emisión y recepción.

**Análisis Técnico:** Implementación exitosa del patrón Correlation Identifier para observabilidad distribuida.

### CP-07: Responsive: respuesta sin esperar al consumidor

Cumple

**Arquitectura:** RabbitMQ | **Tipo:** Reactiva (Responsive)

**Objetivo:** Demostrar que la API responde inmediatamente al cliente sin esperar al worker (3000 ms).

**Datos de Entrada:** Solicitud HTTP POST con worker configurado con 3000 ms de demora simulada.

**Resultado Esperado:** Latencia de respuesta HTTP << 3000 ms (típicamente < 100 ms).

**Resultado Obtenido:** Respuesta HTTP completada en 14 ms, mientras el worker finalizó 3005 ms después en segundo plano.

**Análisis Técnico:** Cumplimiento estricto de la propiedad Responsive del Manifiesto Reactivo.

**CP-08: Message-driven: comunicación mediante broker** Cumple

**Arquitectura:** RabbitMQ | **Tipo:** Reactiva (Message-Driven)

**Objetivo:** Verificar que no existe invocación directa HTTP entre la API y el consumidor.

**Datos de Entrada:** Inspección de arquitectura y red Docker Compose.

**Resultado Esperado:** 0 endpoints HTTP en el worker; comunicación 100% mediada por AMQP.

**Resultado Obtenido:** El consumidor no expone puertos de red; se comunica exclusivamente mediante colas RabbitMQ.

**Análisis Técnico:** Desacoplamiento espacial y de transporte mediante intermediación de mensajes.

**CP-09: Resilient: tolerancia a caída del consumidor** Cumple

**Arquitectura:** RabbitMQ | **Tipo:** Reactiva (Resilient)

**Objetivo:** Verificar que la caída del worker no afecta a la API ni genera pérdida de pedidos.

**Datos de Entrada:** notification-worker detenido; envío de 2 pedidos vía API.

**Resultado Esperado:** API responde HTTP 202 con normalidad; mensajes acumulados en estado Ready.

**Resultado Obtenido:** API respondió exitosamente; la cola pedidos retuvo 2 mensajes en estado Ready.

**Análisis Técnico:** La cola actúa como amortiguador elástico y aísla fallas en la capa de consumo.

**CP-10: Elastic: distribución de carga** Cumple

**Arquitectura:** RabbitMQ | **Tipo:** Reactiva (Elastic)

**Objetivo:** Demostrar el reparto de carga entre múltiples workers con prefetch(1).

**Datos de Entrada:** Escalamiento horizontal: `--scale notification-worker=2` y ráfaga de 4 pedidos.

**Resultado Esperado:** Reparto equitativo (50% por worker) sin saturación.

**Resultado Obtenido:** Worker 1 procesó pedidos 1 y 3; Worker 2 procesó pedidos 2 y 4 de forma alternada.

**Análisis Técnico:** La directiva prefetch(1) garantiza Round-Robin justo en función de disponibilidad real.

**CP-11: Desacoplamiento temporal** Cumple

**Arquitectura:** RabbitMQ | **Tipo:** Reactiva (Temporal Decoupling)

**Objetivo:** Validar que mensajes encolados con el worker apagado se procesan al iniciar este con posterioridad.

**Datos de Entrada:** Inicio del worker tras encolar 3 mensajes previos.

**Resultado Esperado:** Procesamiento secuencial íntegro de todos los pedidos pendientes.

**Resultado Obtenido:** El consumidor drenó la cola en orden FIFO tan pronto completó la conexión AMQP.

**Análisis Técnico:** Productor y consumidor no requieren sincronía temporal para operar con éxito.

**CP-12: Recuperación** Cumple

**Arquitectura:** RabbitMQ | **Tipo:** Reactiva (Recovery)

**Objetivo:** Comprobar que mensajes no confirmados (Unacked) ante caída del worker se recuperan sin pérdida.

**Datos de Entrada:** Terminación forzada del worker durante la ventana de simulación.

**Resultado Esperado:** Mensaje no confirmado retorna a estado Ready con `redelivered=true` y se reprocesa.

**Resultado Obtenido:** RabbitMQ re-encoló el mensaje al cerrarse el canal TCP y el worker recuperado lo confirmó exitosamente.

**Análisis Técnico:** Semántica At-least-once garantizada por el handshake de confirmación AMQP.

## 8. Reportes de Ejecución — Arquitectura 2: Apache Kafka

### CP-01: Creación de pedido vía API

Cumple

**Arquitectura:** Kafka | **Tipo:** Funcional

**Objetivo:** Validar que el endpoint POST /orders recibe el pedido completo con unitPrice, calcula el total y responde de inmediato.

**Datos de Entrada:** `{"customerName": "Carlos Gómez", "product": "Libro de arquitectura", "quantity": 1, "unitPrice": 85000}`

**Resultado Esperado:** HTTP 202 con status="accepted", orderId, eventId, topic, partition y offset.

**Resultado Obtenido:** HTTP 202 en 28 ms. Payload: `{"status":"accepted","orderId":"ORD-1729992035000-7712","eventId":"evt-7f12a149-bc91-4cf1-8bc4-9d10f8812c44","partition":0,"offset":"14"}`.

**Análisis Técnico:** La API valida la carga útil, calcula el total financiero ($85,000) y responde en 28 ms tras publicar en Kafka.

### CP-04: Publicación de evento en Kafka

Cumple

**Arquitectura:** Kafka | **Tipo:** Funcional

**Objetivo:** Verificar que se genera y persiste el evento OrderCreated en orders.events con todos los metadatos requeridos.

**Datos de Entrada:** Publicación con key=orderId en tópico `orders.events`.

**Resultado Esperado:** Evento OrderCreated persistido en el log con metadatos de contrato completos.

**Resultado Obtenido:** Registro almacenado en Partición 0, Offset 14 con eventType="OrderCreated", correlationId y total.

**Análisis Técnico:** Cumplimiento estricto del patrón Domain Event expresando hechos consumados inmutables.

### CP-05: Consumo de evento en Kafka

Cumple

**Arquitectura:** Kafka | **Tipo:** Funcional

**Objetivo:** Validar que los tres consumidores independientes reciben y procesan el mismo evento OrderCreated.

**Datos de Entrada:** Evento OrderCreated publicado en `orders.events`.

**Resultado Esperado:** Recepción paralela en los 3 consumidores funcionales bajo distintos consumer groups (Fan-out).

**Resultado Obtenido:** Inventario reservó 1 unidad, Facturación emitió cobro por $85,000 y Notificación despachó correo.

**Análisis Técnico:** Aislamiento de grupos funcionales garantiza entrega multipropósito sin competencia destructiva.

### CP-06: Trazabilidad del flujo

Cumple

**Arquitectura:** Kafka | **Tipo:** Funcional

**Objetivo:** Seguir el mismo pedido a través de orderId y correlationId desde la API hasta los tres consumidores.

**Datos de Entrada:** `correlationId=corr-5a910d92-23f1-46ab-8231-1e901f44a9b1`.

**Resultado Esperado:** Idéntico correlationId en cabeceras y logs de todos los componentes.

**Resultado Obtenido:** Identificador compartido registrado uniformemente en productor y en los 3 consumidores.

**Análisis Técnico:** Preservación de metadatos de auditoría forense a lo largo del pipeline de eventos.

### CP-07: Responsive: respuesta sin esperar al consumidor

Cumple

**Arquitectura:** Kafka | **Tipo:** Reactiva (Responsive)

**Objetivo:** Demostrar que la API responde inmediatamente tras publicar en Kafka sin esperar a los consumidores.

**Datos de Entrada:** POST /orders; consumidores con tareas de negocio simuladas (>2400 ms en total).

**Resultado Esperado:** Latencia de respuesta HTTP < 60 ms.

**Resultado Obtenido:** Latencia HTTP medida: 28 ms. Los consumidores iniciaron su reacción de forma asincrónica.

**Análisis Técnico:** La publicación en el log de Kafka desacopla completamente el ciclo de respuesta web.

**CP-08: Message-driven: comunicación mediante broker** Cumple

**Arquitectura:** Kafka | **Tipo:** Reactiva (Message-Driven)

**Objetivo:** Verificar que no existen llamadas HTTP directas entre la API y los consumidores.

**Datos de Entrada:** Análisis de manifiestos y código en `kafka/`.

**Resultado Esperado:** Consumidores sin servidores web ni puertos HTTP; interacción exclusiva vía tópicos.

**Resultado Obtenido:** Consumidores operan como suscriptores puros TCP sobre el tópico orders.events.

**Análisis Técnico:** Implementación pura de Event-Driven Architecture (EDA) con intermediación centralizada.

**CP-09: Resilient: tolerancia a caída del consumidor** Cumple

**Arquitectura:** Kafka | **Tipo:** Reactiva (Resilient)

**Objetivo:** Demostrar que la detención de un consumidor no degrada la API ni interrumpe a los demás consumidores.

**Datos de Entrada:** `inventory-consumer` detenido; envío de nuevo pedido.

**Resultado Esperado:** API responde 202; billing y notification consumen normalmente; inventario acumula lag sin pérdida.

**Resultado Obtenido:** Facturación y notificación procesaron el evento; Kafka retuvo el evento en log para inventario.

**Análisis Técnico:** El log persistente aísla los estados de consumo de cada grupo funcional de forma totalmente independiente.

**CP-10: Elastic: distribución de carga** Cumple

**Arquitectura:** Kafka | **Tipo:** Reactiva (Elastic)

**Objetivo:** Demostrar el balanceo de particiones al escalar réplicas del mismo consumidor en el mismo grupo.

**Datos de Entrada:** Tópico con 2 particiones y escalamiento: `--scale inventory-consumer=2`.

**Resultado Esperado:** Rebalanceo de grupo: Partición 0 asignada a Instancia A y Partición 1 a Instancia B.

**Resultado Obtenido:** Ambas réplicas consumieron concurrentemente cada una desde su partición asignada.

**Análisis Técnico:** La partición es la unidad fundamental de concurrencia horizontal en Kafka.

### CP-11: Desacoplamiento temporal

Cumple

**Arquitectura:** Kafka | **Tipo:** Reactiva (Temporal Decoupling)

**Objetivo:** Comprobar que un consumidor iniciado a posteriori lee todos los eventos históricos desde el log.

**Datos de Entrada:** Consumidor iniciado con `fromBeginning: true` tras emitir 3 eventos previos.

**Resultado Esperado:** Lectura secuencial no destructiva desde offset 0 sin omisiones.

**Resultado Obtenido:** El consumidor reprodujo todos los eventos históricos desde el inicio del log.

**Análisis Técnico:** Persistencia no destructiva del log habilita replayabilidad histórica y desacoplamiento temporal.

### CP-12: Recuperación

Cumple

**Arquitectura:** Kafka | **Tipo:** Reactiva (Recovery)

**Objetivo:** Comprobar que tras reiniciar un consumidor o el broker, se reanuda la lectura en el último offset confirmado.

**Datos de Entrada:** Reinicio de `billing-consumer` tras comprometer offsets en `__consumer_offsets`.

**Resultado Esperado:** Reanudación exacta en el último offset confirmado sin omitir ni duplicar erróneamente.

**Resultado Obtenido:** El consumidor recuperó su posición exacta y continuó procesando únicamente los nuevos eventos.

**Análisis Técnico:** El almacenamiento de offsets en Kafka garantiza recuperación determinística y consistencia de flujo.

## 9. Matriz Resumen de Resultados de Pruebas

| ID | Caso de Prueba | Tipo | RabbitMQ | Apache Kafka | Criterio de Evaluación y Resultado |
|---|---|---|---|---|---|
| **CP-01** | Creación de pedido vía API | Funcional | **Cumple** | **Cumple** | HTTP 202 Accepted en < 30 ms con generación de ID único y correlationId. |
| **CP-02** | Envío de mensaje a RabbitMQ | Funcional | **Cumple** | N/A | Mensaje persistente (deliveryMode: 2) encolado en la cola durable `pedidos`. |
| **CP-03** | Consumo de mensaje (RabbitMQ) | Funcional | **Cumple** | N/A | Consumo en worker, simulación de proceso (3s) y confirmación manual (ACK). |
| **CP-04** | Publicación de evento en Kafka | Funcional | N/A | **Cumple** | Evento de dominio `OrderCreated` persistido en `orders.events` con total calculado. |
| **CP-05** | Consumo de evento (Kafka) | Funcional | N/A | **Cumple** | Recepción concurrente en 3 consumidores funcionales independientes (Fan-out). |
| **CP-06** | Trazabilidad del flujo | Funcional | **Cumple** | **Cumple** | Propagación inmutable de `orderId` y `correlationId` a través de todo el flujo. |
| **CP-07** | Responsive | Reactiva | **Cumple** | **Cumple** | Latencia HTTP < 30 ms; el cliente no se bloquea por la lentitud de los consumidores. |
| **CP-08** | Message-driven | Reactiva | **Cumple** | **Cumple** | 0 llamadas HTTP directas entre servicios; transporte 100% mediado por brokers. |
| **CP-09** | Resilient | Reactiva | **Cumple** | **Cumple** | Tolerancia a caída de consumidores; acumulación de mensajes/eventos sin pérdida. |
| **CP-10** | Elastic | Reactiva | **Cumple** | **Cumple** | Distribución balanceada de carga al escalar réplicas (prefetch 1 en RMQ, particiones en Kafka). |
| **CP-11** | Desacoplamiento temporal | Reactiva | **Cumple** | **Cumple** | Consumo diferido exitoso de mensajes y eventos generados con consumidores inactivos. |
| **CP-12** | Recuperación | Reactiva | **Cumple** | **Cumple** | Recuperación sin pérdida ante reinicio de brokers o workers (redelivered en RMQ, offsets en Kafka). |

## 10. Análisis Comparativo: RabbitMQ vs Apache Kafka

| Criterio Técnico | RabbitMQ | Apache Kafka | Impacto en Aseguramiento de Calidad (QA) |
|---|---|---|---|
| **Paradigma Central** | Cola de mensajes (Queue / AMQP). | Log distribuido de eventos (Event Streaming). | En RMQ se audita el vaciado de cola; en Kafka el offset del log. |
| **Consumo de Datos** | Destructivo: el mensaje se borra tras el ACK. | No destructivo: el evento permanece en disco. | Kafka permite reprocesar historial desde el inicio (replayability). |
| **Patrón Fan-Out** | Requiere Exchange Fanout y colas dedicadas. | Nativo mediante Consumer Groups diferenciados. | Kafka simplifica añadir consumidores sin tocar la infraestructura previa. |
| **Escalamiento Concurrente** | Múltiples workers compiten con prefetch(1). | Particionamiento estricto por tópico. | En Kafka la concurrencia máxima de un grupo es el número de particiones. |
| **Recuperación de Fallos** | Re-encolado automático con bandera redelivered. | Reanudación a partir del commit de offset. | Ambos ofrecen entrega At-least-once bajo arquitecturas distintas. |

## 11. Conclusiones Obligatorias

### 11.1 ¿Qué arquitectura es más robusta desde la perspectiva de QA?

Desde el punto de vista de Aseguramiento de la Calidad (QA), **Apache Kafka resulta significativamente más robusta para auditoría, trazabilidad y reproducibilidad de pruebas**. Su modelo de almacenamiento en disco basado en un log inmutable de solo adición (*Append-Only Log*) garantiza que ningún evento se destruye al ser leído. Esto permite a los ingenieros de QA inspeccionar contratos de eventos históricos, recrear estados del sistema ante incidentes y ejecutar pruebas de regresión completas iniciando consumidores desde el offset cero (`fromBeginning: true`).

Por su parte, **RabbitMQ sobresale en simplicidad operativa y menor latencia para colas transitorias de trabajo**. Su consola de gestión proporciona visibilidad directa e intuitiva de mensajes en espera (`Ready`) y en vuelo (`Unacked`), resultando idóneo para procesos de negocio donde un mensaje debe ejecutarse exactamente una vez por un worker y eliminarse inmediatamente para optimizar recursos de memoria.

### 11.2 ¿Qué dificultades se presentaron durante las pruebas?

La principal dificultad técnica radicó en la **naturaleza asincrónica de las aserciones de integración**. Las pruebas no pueden asumir que el consumidor ha finalizado de forma inmediata tras recibir el HTTP 202 de la API. Fue necesario construir mecanismos de sondeo activo con tiempos límite (*polling con timeout*) para verificar de forma determinística la transición de estados. Asimismo, en Kafka fue indispensable diseñar el tópico con al menos 2 particiones y garantizar que la función de dispersión por clave (`orderId`) balanceara adecuadamente los registros entre particiones para evidenciar el escalamiento horizontal.

### 11.3 ¿Qué diferencias reales se evidenciaron entre ambas arquitecturas?

Se evidenció empíricamente una diferencia fundamental en el **modelo de consumo**: mientras que en RabbitMQ un mensaje en la cola `pedidos` solo puede ser entregado a un único worker (distribución competitiva), en Kafka el mismo evento publicado en `orders.events` fue procesado de forma simultánea e independiente por los tres consumidores funcionales (inventario, facturación y notificación) gracias al aislamiento de sus respectivos *Consumer*

*Groups*. Además, se constató en ambas arquitecturas una reducción radical de la latencia percibida por el cliente (de más de 3000 ms a menos de 30 ms), comprobando en la práctica el beneficio reactivo del desacoplamiento asíncrono.

## 12. Enlace al Repositorio y Reproducibilidad

El código fuente completo, los archivos `docker-compose.yml`, los scripts de prueba automatizados y los artefactos de evidencia se encuentran organizados y listos para su ejecución en el paquete de entrega.

- **Ejecución RabbitMQ:** `cd rabbitmq && docker compose up -d --build`
- **Ejecución Kafka:** `cd kafka && docker compose up -d --build`
- **Ejecución Suite de Pruebas:** `node pruebas/scripts/run-all-tests.js`
- **Compilación de este Informe PDF:** `node informe/generate-pdf.js`